

Министерство цифрового развития государственного управления,
информационных технологий и связи Республики Татарстан
Государственное автономное профессиональное образовательное
учреждение «Международный центр компетенций – Казанский техникум
информационных технологий и связи»

Курсовой проект выполнен
и защищен с оценкой _____
Руководитель КП _____
«__» _____ 20__ г

КУРСОВОЙ ПРОЕКТ

по МДК 01.01 Разработка программных модулей
на тему: Разработка WEB приложения для угадывания слов «GuessWord»

Руководитель

(подпись и дата руководителя)

А.В.Фролов

Исполнитель,
студент группы 320П

(подпись и дата исполнителя)

В.В.Смирнов

Казань, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО ПРОДУКТА	5
1.1 Диаграмма USE CASE	5
1.2 Функциональные требования	6
1.3 Проектирование архитектуры приложения	7
2 ОБОСНОВАНИЕ ВЫБОРА СРЕДСТВ РАЗРАБОТКИ	9
2.1 Язык разработки	9
2.2 Технологии разработки	10
2.3 Выбор СУБД	11
2.4 Среда разработки	12
2.5 Средства проектирования интерфейса	13
3 РАЗРАБОТКА ПРОГРАММНОГО ПРОДУКТА	15
3.1 Разработка макета пользовательского интерфейса	15
3.2 Проектирование базы данных	17
3.3 Разработка серверной части	19
3.4 Разработка клиентской части	22
3.5 Реализация игровой логики	25
3.6 Развертывание веб-приложения	28
4 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ	30
4.1 Использование программы	30
5 ТЕСТИРОВАНИЕ	34
5.1 Тестирование методом «черного ящика»	34
5.2 Тестирование методом «белого ящика»	35
ЗАКЛЮЧЕНИЕ	36
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ	37
ПРИЛОЖЕНИЕ А	38
ПРИЛОЖЕНИЕ В	44

ВВЕДЕНИЕ

В настоящее время веб-приложения занимают важное место в сфере развлечений, обучения и организации совместного досуга пользователей. Особенно востребованы интерактивные сервисы, позволяющие совмещать игровой процесс с интеллектуальной активностью, соревновательным элементом и взаимодействием между участниками в режиме реального времени. Одним из таких направлений являются словесные игры, в которых пользователь должен находить или угадывать скрытое слово на основе получаемых подсказок.

Актуальность разработки подобного программного продукта связана с ростом интереса к браузерным многопользовательским приложениям, не требующим установки дополнительного программного обеспечения. Использование современных веб-технологий позволяет запускать такие системы на различных устройствах, обеспечивать удобный доступ через интернет и поддерживать одновременную работу большого количества пользователей.

Целью данной курсовой работы является разработка веб-приложения GuessWord, предназначенного для угадывания слов в одиночном и многопользовательском режимах, с поддержкой авторизации пользователей, ведения игровой статистики и взаимодействия участников через комнаты.

Для достижения поставленной цели необходимо решить следующие задачи:

- проанализировать особенности предметной области словесных онлайн-игр;
- определить функциональные требования к разрабатываемой системе;
- спроектировать структуру базы данных для хранения пользователей, игр, комнат и результатов матчей;
- спроектировать архитектуру клиентской и серверной частей приложения;
- разработать удобный пользовательский интерфейс веб-приложения;
- реализовать механизм регистрации и авторизации пользователей с использованием JWT;
- реализовать одиночный режим игры;

- разработать многопользовательский режим с использованием комнат и технологии SignalR;
- реализовать хранение истории игр и таблицу лидеров;
- внедрить механизм подбора слов на основе embeddings и косинусной близости;
- выполнить тестирование разработанного приложения и устранение выявленных ошибок.

В качестве средств разработки используются современные технологии платформы .NET. Серверная часть реализована на ASP.NET Core Web API, клиентская часть разработана с использованием Blazor WebAssembly. Для хранения данных применяется PostgreSQL, а взаимодействие с базой данных организовано через Entity Framework Core. Для обмена данными в режиме реального времени используется SignalR.

Особенностью проекта является применение в игровой логике векторных представлений слов, позволяющих определять степень смысловой близости между введенным словом и загаданным вариантом. Это делает игровой процесс более гибким и интересным, поскольку пользователь получает не только информацию о правильности ответа, но и оценку близости попытки к целевому слову.

В итоге разработка веб-приложения GuessWord направлена на создание современного интерактивного сервиса, сочетающего игровые механики, многопользовательское взаимодействие и применение актуальных технологий веб-разработки.

1 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО ПРОДУКТА

1.1 Диаграмма USE CASE

Диаграмма вариантов использования (Use Case Diagram) применяется для описания основных возможностей программного продукта с точки зрения пользователя. Она позволяет определить доступные действия в системе, сценарии, требующие авторизации, и способы взаимодействия пользователя с веб-приложением.

В веб-приложении GuessWord основным актором является пользователь. Для данной роли определен набор основных сценариев взаимодействия с системой. Пользователь может перейти на главную страницу, зарегистрироваться или пройти авторизацию для получения доступа к игровому функционалу.

После входа в систему пользователю становятся доступны запуск одиночной и многопользовательской игры, а также участие в игровом процессе путем ввода слов-попыток. Кроме того, предусмотрены просмотр профиля, истории сыгранных матчей и таблицы лидеров. Диаграмма вариантов использования для данной роли представлена на рисунке 1.1.

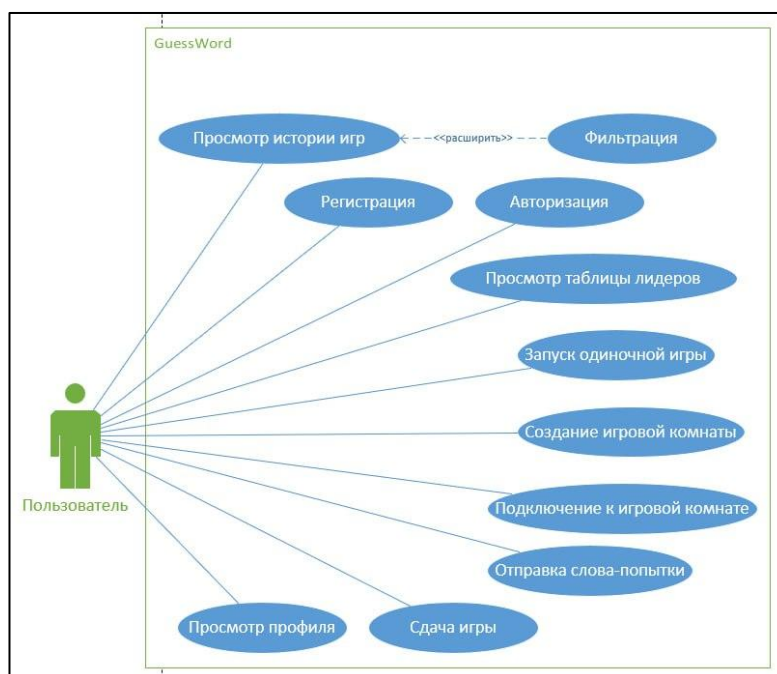


Рисунок 1.1 Диаграмма USE CASE для пользователя

Одиночная игра представляет собой режим, в котором пользователь пытается угадать загаданное слово.

После каждой попытки система обрабатывает введенное слово и возвращает его ранг относительно загаданного, рассчитанный на основе смысловой близости. Это позволяет постепенно приближаться к правильному ответу, анализируя предыдущие результаты.

Многопользовательская игра связана с использованием комнат. Пользователь может создать комнату и ожидать второго участника либо присоединиться к существующей по специальному коду. После начала матча игроки соревнуются между собой, а обмен игровыми событиями выполняется в режиме реального времени.

Отдельными вариантами использования являются просмотр истории игр и таблицы лидеров. История игр содержит сведения о ранее сыгранных матчах с возможностью фильтрации по типу игры, а таблица лидеров отображает результаты игроков и поддерживает соревновательный элемент.

Для работы большинства функций требуется авторизация, поскольку системе необходимо связывать результаты игр, историю матчей и рейтинг с конкретной учетной записью. Регистрация и вход являются базовыми сценариями, от которых зависит доступ к основному функционалу приложения.

В итоге диаграмма вариантов использования определяет границы функциональности GuessWord, отражает основные сценарии взаимодействия пользователя с системой и упрощает дальнейшее проектирование клиентской части, серверной логики и базы данных.

1.2 Функциональные требования

Функциональные требования определяют, какие возможности должно предоставлять веб-приложение пользователю и какие действия система должна выполнять в процессе работы. Для проекта GuessWord требования связаны с регистрацией пользователей, организацией игрового процесса, хранением результатов и поддержкой многопользовательского режима.

Веб-приложение должно обеспечивать регистрацию и авторизацию пользователей.

При создании учетной записи введенные данные сохраняются в базе данных, а авторизация предоставляет доступ к основному функционалу системы. Для защиты пользовательских данных и ограничения доступа используется механизм JWT-токенов.

После авторизации пользователь должен иметь возможность запускать одиночную игру. В данном режиме система выбирает загаданное слово, а пользователь вводит варианты ответа. Каждая попытка обрабатывается серверной частью, после чего возвращается результат, показывающий степень близости введенного слова к загаданному. Для этого используется сравнение слов на основе embeddings и косинусной близости.

Также в приложении реализован многопользовательский режим. Пользователь может создать игровую комнату или подключиться к существующей по коду. После подключения участников система обеспечивает обмен игровыми событиями в режиме реального времени. Для этого используется технология SignalR.

Система должна сохранять сведения о завершенных играх. История игр позволяет просматривать данные о ранее сыгранных матчах, что необходимо для отслеживания прогресса пользователя.

Отдельным требованием является формирование таблицы лидеров. Приложение должно отображать рейтинг пользователей на основе результатов игр, что усиливает соревновательный элемент.

Таким образом, перечисленные требования определяют основной набор функций, необходимый для корректной работы GuessWord, включая одиночный режим, многопользовательское взаимодействие, хранение статистики и обработку игровой логики на стороне сервера.

1.3 Проектирование архитектуры приложения

Архитектура веб-приложения GuessWord спроектирована с разделением на клиентскую и серверную части. Такой подход упрощает разработку, поддержку системы и дальнейшее расширение функционала.

Клиентская часть реализована с использованием Blazor WebAssembly [3]. Она отвечает за отображение страниц, обработку действий пользователя и отправку запросов к серверу. Через клиентский интерфейс пользователь проходит регистрацию и авторизацию, запускает одиночную игру, создает комнату, подключается к многопользовательскому режиму, просматривает историю игр и таблицу лидеров.

Серверная часть построена на ASP.NET Core Web API [2]. Она принимает запросы клиента, выполняет проверку данных, обрабатывает игровую логику, взаимодействует с базой данных и возвращает результаты. Для защиты отдельных методов используется авторизация на основе JWT-токенов.

Для хранения данных применяется база данных PostgreSQL [4]. В ней сохраняется информация о пользователях, игровых комнатах, играх, участниках, попытках и словах. Работа с базой данных организована через Entity Framework Core, что упрощает выполнение операций с данными.

Отдельное место в архитектуре занимает технология SignalR, используемая в многопользовательском режиме. Она обеспечивает обмен данными между участниками комнаты в режиме реального времени без обновления страницы.

Взаимодействие компонентов происходит следующим образом: пользователь выполняет действие в интерфейсе, клиентская часть отправляет запрос к серверному API, сервер обрабатывает его, при необходимости обращается к базе данных и возвращает результат. В многопользовательском режиме часть событий передается через SignalR Hub для синхронизации игроков.

Выбранная архитектура соответствует задачам проекта и позволяет реализовать одиночную игру, многопользовательские комнаты, авторизацию, историю игр, таблицу лидеров и обработку слов с использованием embeddings и косинусной близости.

2 ОБОСНОВАНИЕ ВЫБОРА СРЕДСТВ РАЗРАБОТКИ

2.1 Язык разработки

Для разработки веб-приложения GuessWord выбран язык программирования C# [1]. Он является основным для платформы .NET и широко используется при создании веб-приложений и клиент-серверных систем. Его применение позволило реализовать серверную и клиентскую части проекта в рамках единой технологической среды.

Одним из преимуществ C# является строгая типизация, которая помогает выявлять ошибки на ранних этапах разработки и повышает надежность программного кода. Это особенно важно для приложений с большим количеством сущностей, связей между моделями данных и пользовательских сценариев.

Язык поддерживает объектно-ориентированный подход, что удобно при описании структуры проекта. В GuessWord это используется при работе с сущностями пользователей, игровых комнат, игр, попыток и слов. Применение классов и интерфейсов упрощает организацию и сопровождение кода.

Важным преимуществом C# является поддержка асинхронного программирования с использованием `async` и `await`. Это особенно полезно для веб-приложений при работе с базой данных, обработке HTTP-запросов и обмене данными между клиентом и сервером.

Использование C# позволило применять единый стек технологий: серверная часть реализована на ASP.NET Core Web API, а клиентская — на Blazor WebAssembly, где также используется данный язык. Это избавляет от необходимости применять разные языки для frontend и backend.

Дополнительным преимуществом является развитая экосистема платформы .NET. Для C# доступны библиотеки для работы с базами данных, авторизацией, сериализацией данных и сетевым взаимодействием. В проекте GuessWord это позволило использовать Entity Framework Core, JWT-аутентификацию и SignalR.

В итоге выбор языка C# является обоснованным, так как он обеспечивает удобство разработки, надежность кода и тесную интеграцию с технологиями платформы .NET, используемыми в проекте GuessWord.

2.2 Технологии разработки

Для реализации веб-приложения GuessWord выбраны современные технологии платформы .NET, позволяющие создать интерактивный и удобный программный продукт. Выбор средств разработки осуществлялся с учетом особенностей проекта, включающего клиентскую и серверную части, авторизацию пользователей, работу с базой данных и многопользовательский режим.

Клиентская часть приложения разработана с использованием Blazor WebAssembly. Данная технология позволяет создавать веб-интерфейсы на языке C#, выполняемые непосредственно в браузере пользователя. Это дает возможность использовать единый язык программирования для клиентской и серверной части. Также Blazor WebAssembly обеспечивает динамическое обновление интерфейса без полной перезагрузки страниц.

Серверная часть построена на основе ASP.NET Core Web API. Данная технология используется для обработки HTTP-запросов, выполнения бизнес-логики и взаимодействия с базой данных. Через API клиентское приложение получает данные, отправляет игровые действия и выполняет авторизацию.

Для работы с базой данных применяется Entity Framework Core [5]. Использование ORM-подхода позволило описывать структуру данных через классы C# и выполнять операции чтения, добавления, изменения и удаления записей средствами языка программирования. Это упрощает сопровождение проекта.

Для многопользовательского режима используется SignalR [6], обеспечивающий обмен сообщениями между клиентами и сервером в режиме реального времени. Благодаря этому пользователи видят изменения состояния игры сразу после действий соперника.

Механизм аутентификации реализован с использованием JWT (JSON Web Token) [7]. После успешного входа пользователь получает токен, который применяется при обращении к защищенным методам API. Это позволяет организовать безопасный доступ к функциям приложения.

Дополнительно используются HTML и CSS, необходимые для разметки страниц и оформления пользовательского интерфейса.

В итоге выбранный набор технологий соответствует задачам проекта GuessWord и позволил реализовать веб-приложение с интерактивным интерфейсом, серверной логикой, многопользовательским режимом и системой авторизации.

2.3 Выбор СУБД

Для хранения данных веб-приложения GuessWord выбрана система управления базами данных PostgreSQL. Данная СУБД является одной из наиболее распространенных реляционных систем с открытым исходным кодом и широко используется при разработке современных веб-приложений.

Выбор PostgreSQL обусловлен высокой надежностью, стабильностью работы и широкими функциональными возможностями. Система поддерживает корректную обработку параллельных запросов, транзакции и механизмы контроля целостности данных. Это особенно важно для проекта GuessWord, где одновременно могут выполняться регистрация пользователей, создание игровых комнат, сохранение результатов матчей и обработка попыток игроков.

В базе данных проекта требуется хранить сведения о пользователях, игровых комнатах, завершенных играх, участниках матчей и введенных словах. Реляционная модель PostgreSQL хорошо подходит для хранения подобных взаимосвязанных сущностей, так как позволяет использовать первичные и внешние ключи, ограничения целостности и индексацию данных.

Одним из преимуществ PostgreSQL является возможность расширения функциональности через дополнительные модули. В GuessWord используется работа с embeddings и вычислением косинусной близости между словами.

Для подобных задач PostgreSQL подходит благодаря поддержке расширений для работы с векторными типами данных, например pgvector [15]. Это позволяет хранить векторные представления слов в базе данных и выполнять поиск близких значений.

Также PostgreSQL хорошо интегрируется с платформой .NET и поддерживается библиотекой Npgsql, что обеспечивает стабильное подключение из ASP.NET Core и корректную работу через Entity Framework Core.

В итоге выбор PostgreSQL является обоснованным решением для проекта GuessWord, так как данная СУБД обеспечивает надежное хранение данных, удобную работу с реляционной структурой проекта и поддержку векторного поиска.

2.4 Среда разработки

В качестве основной среды разработки для создания веб-приложения GuessWord выбрана интегрированная среда Visual Studio [9]. Данное решение является одним из наиболее распространенных инструментов для разработки программного обеспечения на платформе .NET и предоставляет полный набор средств, необходимых для создания клиент-серверных приложений.

Выбор Visual Studio обусловлен тесной интеграцией со стеком технологий, используемым в проекте. Среда разработки обеспечивает удобную работу с ASP.NET Core Web API, Blazor WebAssembly, Entity Framework Core и другими компонентами платформы .NET. Это позволяет разрабатывать все части приложения в едином рабочем пространстве без необходимости использования большого количества сторонних инструментов.

Одним из ключевых преимуществ Visual Studio является развитый редактор исходного кода. Он поддерживает подсветку синтаксиса, автоматическое завершение кода, навигацию по проекту, рефакторинг и встроенную проверку ошибок. За счет этого ускоряется написание программного кода и снижается вероятность появления синтаксических и логических ошибок.

Среда разработки предоставляет встроенные средства отладки приложения. Это позволяет пошагово выполнять код, отслеживать значения переменных, анализировать исключения и проверять корректность работы отдельных модулей системы. Данные возможности особенно важны при разработке игровой логики, авторизации пользователей и взаимодействия клиентской части с сервером

Для работы с базой данных в Visual Studio удобно использовать инструменты управления пакетами NuGet и консоль команд Entity Framework Core. С их помощью можно подключать необходимые библиотеки, создавать миграции и обновлять структуру базы данных.

Дополнительным преимуществом является удобная организация структуры решения. В рамках одного решения можно разместить серверный проект, клиентский проект и общие библиотеки с моделями данных. Это упрощает сопровождение проекта GuessWord и обеспечивает логичное разделение компонентов системы.

В итоге использование Visual Studio позволило организовать эффективный процесс разработки приложения GuessWord, ускорить написание кода, упростить тестирование и обеспечить удобную работу со всеми используемыми технологиями проекта.

2.5 Средства проектирования интерфейса

При разработке веб-приложения GuessWord особое внимание уделялось выбору средств проектирования пользовательского интерфейса. От удобства используемых инструментов зависит скорость разработки, качество визуального оформления и возможность предварительно продумать структуру будущего приложения.

Для проектирования внешнего вида страниц и предварительной проработки интерфейса выбран графический редактор Figma [8]. Данный инструмент широко применяется при создании веб-интерфейсов и позволяет разрабатывать макеты страниц до начала программной реализации.

Использование Figma дает возможность заранее определить расположение элементов управления, структуру экранов, цветовую схему и общую стилистику приложения.

Преимуществом Figma является удобство внесения изменений на раннем этапе разработки. При необходимости можно быстро изменить расположение блоков, размеры элементов, типографику и структуру навигации без изменения программного кода. Это позволяет сократить количество доработок на этапе реализации и заранее оценить удобство пользовательского взаимодействия.

Для непосредственной реализации интерфейса в рабочем приложении выбрана технология Blazor WebAssembly. Она позволяет создавать интерактивные пользовательские страницы средствами языка C#, что особенно удобно при использовании единого стека технологий платформы .NET.

Применение Blazor WebAssembly позволяет объединить визуальную часть приложения и программную логику компонентов в единой среде разработки. За счет этого упрощается сопровождение проекта, ускоряется создание интерфейса и обеспечивается тесная интеграция с серверной частью приложения.

Дополнительно для оформления страниц используются HTML и CSS, обеспечивающие структуру элементов и их визуальное представление. С помощью данных технологий настраивается расположение блоков, размеры компонентов, цветовое оформление, адаптация интерфейса и единый стиль приложения.

В итоге выбранный набор инструментов позволил последовательно перейти от проектирования макета к созданию полноценного пользовательского интерфейса веб-приложения GuessWord. Использование Figma упростило подготовительный этап, а Blazor WebAssembly обеспечил удобную реализацию интерфейса в готовом программном продукте.

3 РАЗРАБОТКА ПРОГРАММНОГО ПРОДУКТА

3.1 Разработка макета пользовательского интерфейса

На начальном этапе разработки веб-приложения GuessWord создан предварительный макет пользовательского интерфейса (Рисунок 3.1). Целью данного этапа являлось определение общего визуального стиля приложения, структуры основных страниц и расположения элементов управления до начала программной реализации клиентской части.



Рисунок 3.1 Макет веб-приложения

Разработка макета выполнялась в графическом редакторе Figma. Использование данного инструмента позволило заранее продумать внешний вид приложения, оценить удобство навигации и сформировать единый стиль оформления интерфейса.

На этапе проектирования определялись основные сценарии взаимодействия пользователя с системой: регистрация, запуск игры, просмотр статистики и многопользовательский режим.

Главная страница спроектирована как стартовый экран приложения. На ней размещены краткое описание проекта, переход к игровым режимам и основные разделы системы. Для страниц авторизации и регистрации использовались компактные формы ввода с минимальным количеством полей, что упрощает вход в систему.

Отдельное внимание уделено странице профиля. На ней отображаются основные статистические показатели игрока: количество побед, сыгранных матчей и процент успешных игр. Через профиль пользователь также получает доступ к истории матчей.

Ключевым элементом интерфейса является экран одиночной игры. На макете размещены поле ввода слова, кнопки отправки попытки и очистки результата, а также список предыдущих попыток с отображением степени близости к загаданному слову. Это позволяет быстро анализировать результаты и продолжать игровой процесс.

Для многопользовательского режима создан отдельный экран лобби. На нем размещены элементы создания комнаты, подключения по коду и краткие правила режима игры. Также разработаны страницы истории игр и таблицы лидеров, предназначенные для отображения статистики и результатов игроков.

При создании макетов использовалась единая темная цветовая схема с контрастными акцентами, что обеспечило узнаваемый стиль и хорошую читаемость интерфейса. Основные страницы спроектированы в Figma, дополнительные элементы дорабатывались на этапе реализации клиентской части в Blazor WebAssembly.

В итоге создание макета пользовательского интерфейса позволило заранее определить структуру приложения, сократить количество изменений в процессе разработки и обеспечить единый визуальный стиль GuessWord.

3.2 Проектирование базы данных

Проектирование базы данных являлось одним из ключевых этапов разработки веб-приложения GuessWord, так как от структуры хранения данных зависит корректность работы системы, сохранение игровой статистики и взаимодействие пользователей в многопользовательском режиме.

На начальном этапе создан проект на основе шаблона ASP.NET Core Web API, который послужил основой серверной части приложения. В конфигурационном файле appsettings.json задана строка подключения к базе данных PostgreSQL. Для работы с базой данных подключены пакеты «Microsoft.EntityFrameworkCore», «Microsoft.EntityFrameworkCore.Design», «Npgsql.EntityFrameworkCore.PostgreSQL», «Pgvector» и «Pgvector.EntityFrameworkCore».

Работа с базой данных реализована с использованием Entity Framework Core, что позволило применять подход Code First. При данном подходе структура базы данных формируется на основе моделей данных, созданных в коде приложения.

На начальном этапе определены основные сущности, необходимые для функционирования системы. В проекте GuessWord используются следующие таблицы и связанные с ними модели:

- User - хранение данных зарегистрированных пользователей;
- Room - сведения о созданных игровых комнатах;
- Game - информация о сыгранных партиях;
- GamePlayer - данные об участниках конкретной игры;
- GameAttempt - сведения о попытках угадывания слов;
- Word - словарь слов, используемых в игровом процессе.

Каждая из перечисленных сущностей выполняет отдельную задачу в структуре приложения. Таблица пользователей используется для авторизации, ведения статистики и отображения результатов в таблице лидеров. Таблица комнат необходима для организации многопользовательских матчей и подключения игроков по коду.

Таблица игр содержит сведения о завершенных и текущих партиях, включая режим игры и итоговый результат. Для связи пользователей с конкретной партией используется промежуточная сущность `GamePlayer`, позволяющая учитывать нескольких участников одной игры. Таблица попыток хранит введенные слова, порядок ходов и результаты проверки.

Особое значение имеет таблица `Word`, содержащая словарь допустимых слов. Для записей хранятся векторные представления слов, используемые при вычислении смысловой близости между попыткой игрока и загаданным словом.

После создания моделей настроен контекст базы данных с наборами сущностей `DbSet`. Через него серверная часть выполняет операции чтения, добавления, изменения и удаления данных.

Создание структуры базы данных выполнялось с использованием механизма миграций `Entity Framework Core` (Рисунок 3.2). Это позволило автоматически сформировать таблицы, ключи, связи между сущностями и ограничения целостности данных.

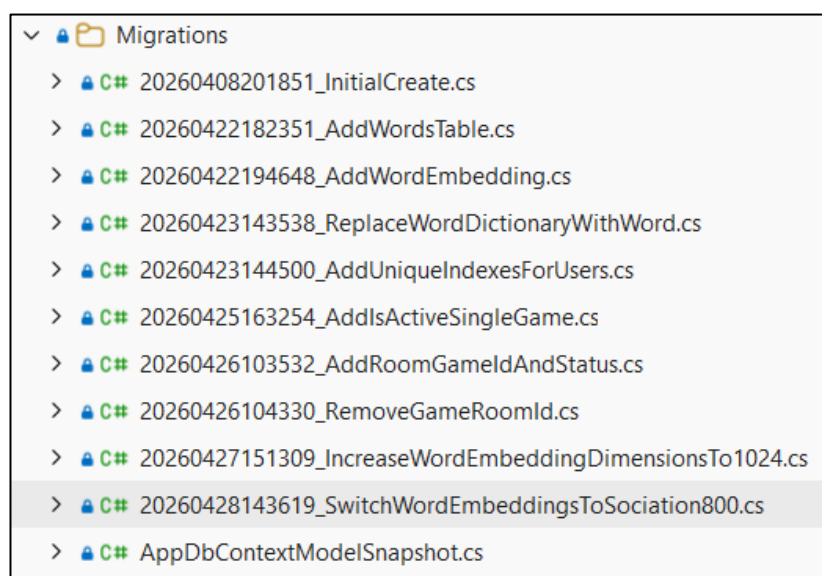


Рисунок 3.2 Список миграций базы данных

При проектировании предусмотрены связи между сущностями. Один пользователь может участвовать во многих играх, одна игра может содержать нескольких участников, каждая партия включает множество попыток угадывания слов. Такое построение структуры обеспечивает корректное хранение истории матчей и статистики пользователей (Рисунок 3.3).

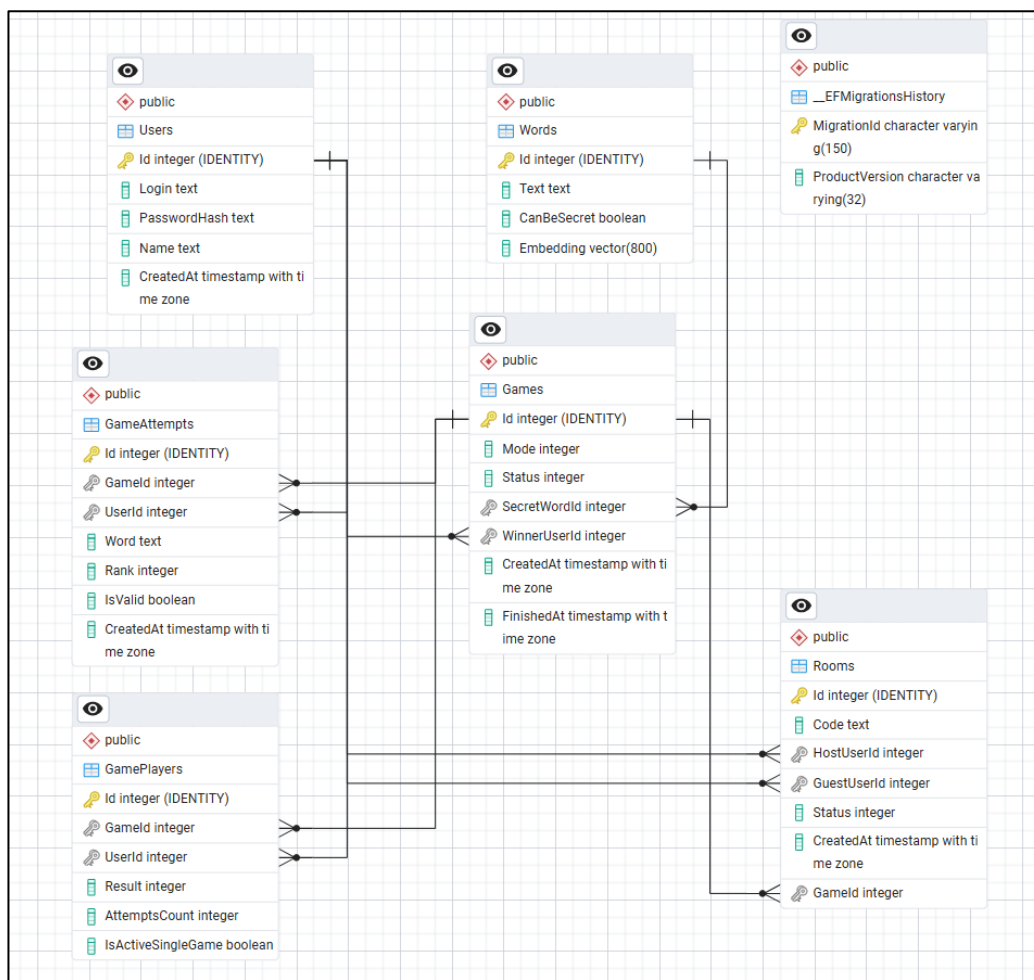


Рисунок 3.3 Схема базы данных

В итоге проектирование базы данных создало основу для стабильной работы приложения GuessWord, включая регистрацию пользователей, сохранение результатов игр, управление комнатами и реализацию игровой логики.

3.3 Разработка серверной части

Разработка серверной части веб-приложения GuessWord выполнена с использованием технологии ASP.NET Core Web API. Серверная часть является центральным элементом системы и отвечает за обработку запросов клиентского приложения, выполнение бизнес-логики и взаимодействие с базой данных.

На начальном этапе создан отдельный серверный проект, в котором выполнена базовая настройка приложения, подключены необходимые библиотеки и настроено взаимодействие с базой данных PostgreSQL через Entity Framework Core.

Для удобства сопровождения проекта структура серверной части разделена на несколько уровней: контроллеры, сервисы, интерфейсы, хабы, модели данных. Также создан отдельный общий проект-библиотека классов, содержащий модели запросов и ответов, используемые серверной и клиентской частью (Рисунок 3.4).

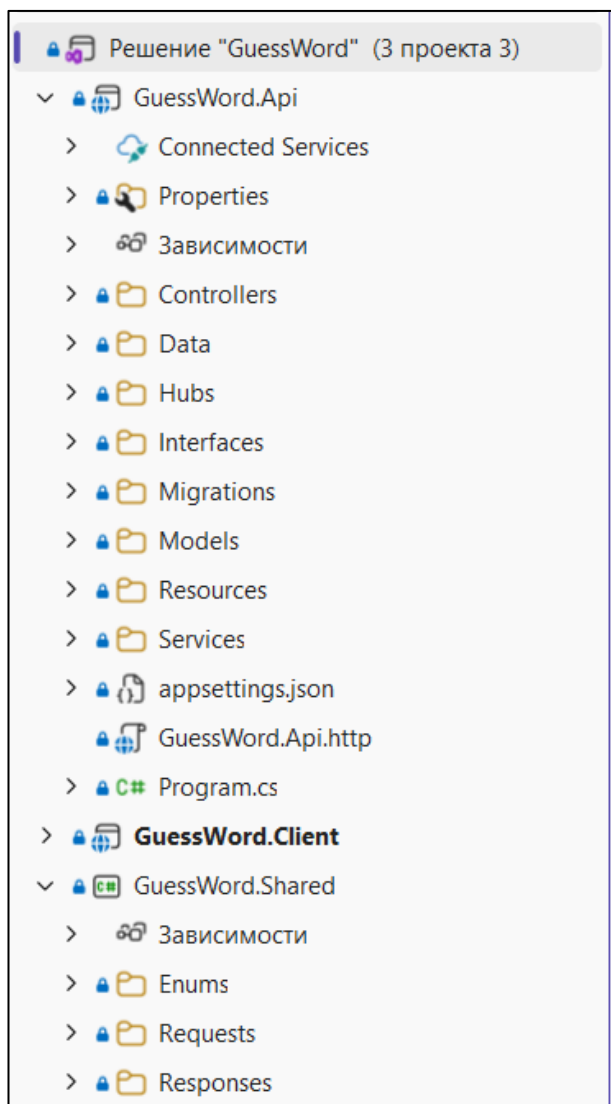


Рисунок 3.4 Структура папок серверной части

Такой подход позволил сделать код более понятным, уменьшить связанность компонентов и упростить дальнейшее расширение функционала.

Контроллеры используются для обработки входящих HTTP-запросов от клиентской части. Каждый контроллер отвечает за отдельное направление работы системы: регистрацию и вход пользователей, игровые действия, работу с комнатами, историей игр и рейтингом игроков. Пример контроллера авторизации представлен на рисунке 3.5.

```
[HttpPost("login")]
Ссылка: 0
public async Task<IActionResult> Login(LoginRequestDto request)
{
    var result = await _userService.Login(request);
    return result is null ? BadRequest("Неверный логин или пароль.") : Ok(result);
}
```

Рисунок 3.5 Контроллер авторизации пользователя

Для отделения входных данных от внутренних моделей приложения использовались классы Request. Они содержат параметры, которые клиент отправляет на сервер. Например, при авторизации пользователь передает логин и пароль, а при создании комнаты — необходимые данные для нового матча.

Для возврата данных клиентской части применялись классы Response. Они содержат результат обработки запроса и только те данные, которые необходимо передать пользователю. Например, после успешного входа система возвращает JWT-токен, имя пользователя и статус операции.

После этого реализован сервисный слой приложения. Сервисы содержат основную бизнес-логику системы: регистрацию пользователей, проверку данных, запуск игр, обработку попыток, сохранение статистики и управление комнатами.

Разделение логики между контроллерами и сервисами позволило оставить контроллеры компактными, а основные вычисления и обработку данных перенести в отдельные классы.

Одной из важнейших частей серверной разработки стала реализация авторизации пользователей. После успешной проверки логина и пароля пользователю выдается JWT-токен, который используется для доступа к защищенным методам API (Рисунок 3.6).

Ссылка: 2

```

public async Task<AuthResponseDto?> Login(LoginRequestDto request)
{
    if (string.IsNullOrEmpty(request.Login) || string.IsNullOrEmpty(request.Password))
        return null;

    var normalizedLogin = request.Login.Trim();

    var user = await _context.Users.FirstOrDefaultAsync(x => x.Login == normalizedLogin);
    if (user is null)
        return null;

    var result = _passwordHasher.VerifyHashedPassword(user, user.PasswordHash, request.Password);
    if (result == PasswordVerificationResult.Failed)
        return null;

    var token = _jwtService.GenerateToken(user);

    return new AuthResponseDto
    {
        UserId = user.Id,
        Token = token,
        Login = user.Login,
        Name = user.Name!
    };
}

```

Рисунок 3.6 Сервис авторизации пользователя

Для игрового процесса разработаны отдельные методы API. Через них клиентское приложение может запускать одиночную игру, отправлять словопопытку, получать результат проверки и завершать матч. Также реализованы методы многопользовательского режима: создание комнаты, подключение по коду, запуск игры и получение обновлений состояния матча.

Для обмена событиями в режиме реального времени подключена технология SignalR, используемая для передачи данных между участниками комнаты без перезагрузки страницы. Дополнительно серверная часть формирует историю игр и таблицу лидеров на основе данных базы.

В итоге серверная часть GuessWord обеспечивает обработку пользовательских запросов, авторизацию, поддержку игровых режимов, взаимодействие с базой данных и синхронизацию игроков в многопользовательском режиме.

3.4 Разработка клиентской части

Разработка клиентской части веб-приложения GuessWord выполнена с использованием технологии Blazor WebAssembly. Клиентская часть отвечает за отображение пользовательского интерфейса, обработку действий пользователя, отправку запросов к серверу и получение данных через Web API.

Структура клиентского проекта включает страницы приложения, компоненты интерфейса, сервисы для взаимодействия с серверной частью и файлы оформления (Рисунок 3.7). Такой подход позволил разделить визуальную часть системы и прикладную логику, связанную с запросами и обработкой данных.

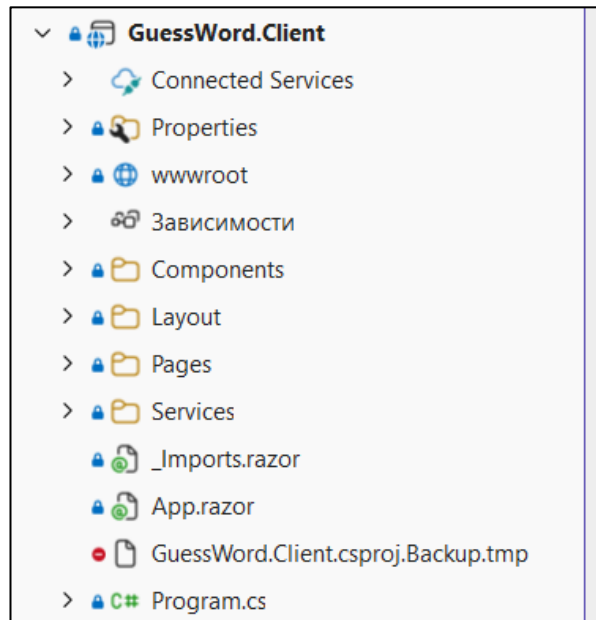


Рисунок 3.7 Структура папок клиентской части

В приложении реализованы основные страницы: главная, авторизация, регистрация, профиль, одиночная игра, создание и подключение к комнате, многопользовательская игра, история игр и таблица лидеров (Рисунок 3.8).

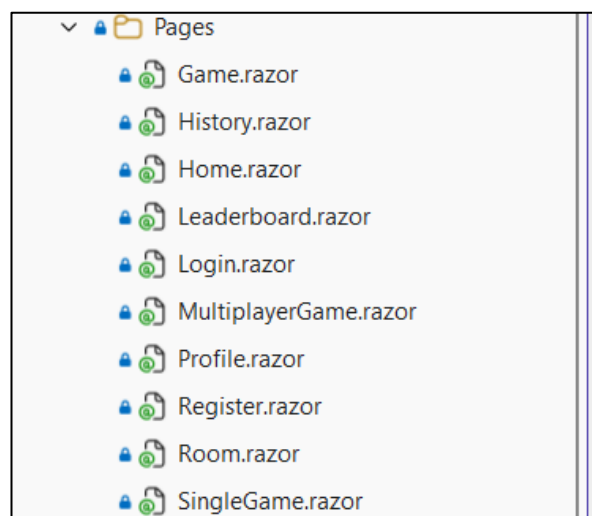


Рисунок 3.8 Список основных страниц сайта

Переход между страницами осуществляется средствами маршрутизации Blazor без полной перезагрузки сайта.

Для обмена данными с серверной частью используется HttpClient. Через него выполняются запросы на регистрацию, вход в систему, запуск игры, отправку попыток, создание комнат, получение истории игр и таблицы лидеров. Для удобства запросы вынесены в отдельные сервисы клиентской части. После получения ответа интерфейс обновляется и отображает актуальные данные.

Для хранения данных авторизации используется локальное хранилище браузера. После успешного входа JWT-токен и основные данные пользователя сохраняются в localStorage. Это позволяет не выполнять повторную авторизацию при обновлении страницы и использовать сохраненный токен при последующих запросах к защищенным методам API.

Также в клиентской части применяется хранение состояния пользователя в отдельном сервисе. Благодаря этому данные об авторизованном пользователе доступны разным страницам приложения без постоянных повторных запросов к серверу.

Игровой режим реализуется через взаимодействие клиента с сервером. При запуске одиночной игры пользователь переходит на соответствующую страницу, вводит слово-попытку и отправляет его на сервер. После обработки запроса система возвращает результат, который отображается в виде списка предыдущих попыток и степени близости введенного слова к правильному ответу.

Для многопользовательского режима пользователь может создать комнату либо подключиться к существующей по коду. После подключения второго игрока начинается матч. Для синхронизации действий используется технология SignalR, благодаря которой клиент получает обновления состояния игры в режиме реального времени без перезагрузки страницы.

Дополнительно реализованы страницы истории игр и таблицы лидеров, на которых отображаются результаты завершенных матчей, количество побед и положение пользователей в рейтинге. Для оформления интерфейса использовались HTML и CSS, что позволило создать единый внешний вид приложения.

В итоге клиентская часть GuessWord обеспечивает регистрацию, авторизацию, выбор режима игры, участие в одиночных и многопользовательских матчах, просмотр статистики и взаимодействие с серверной частью через API и SignalR.

3.5 Реализация игровой логики

Реализация игровой логики являлась ключевым этапом разработки веб-приложения GuessWord, так как именно от нее зависит качество игрового процесса, корректность рангов и общий пользовательский опыт. Основная идея проекта заключается в угадывании скрытого слова не через прямые подсказки, а через определение степени смысловой близости между введенным словом и загаданным вариантом.

Изначально в качестве основы рассматривался подход с использованием обычных embedding-моделей нейросетей. Предполагалось, что для каждого слова можно получить векторное представление, после чего сравнивать слова между собой по косинусной близости. На практике при данном подходе верхняя часть списка ближайших слов выглядела правдоподобно, однако далее появлялись редкие, технические или неестественные для пользователя слова.

После анализа существующих решений пересмотрена вся система построения словаря. В качестве итоговой основы использовалась модель `sociation2vec800.vec`, содержащая ассоциативные векторные представления слов. Данная модель сформирована на основе пользовательских ассоциаций, собранных на сайте `sociation.org` [14], где фиксируются связи между словами на основе ответов реальных людей. В отличие от универсальных `embeddings`, такие векторы лучше подходят для игрового сценария угадывания слов.

Следующим этапом стала подготовка словаря. Исходные данные содержали большое количество лишних словоформ и шумовых элементов, поэтому реализован процесс фильтрации. В итоговом словаре оставлялись преимущественно существительные в именительном падеже и единственном числе.

После очистки словарь разделен на два уровня:

- all-words.txt — полный список допустимых слов, которые пользователь может вводить в игре;
- secret-words.txt — отдельный список слов, которые могут использоваться как загаданные.

Такое разделение необходимо, поскольку не каждое допустимое слово подходит для роли секретного.

Для отбора секретных слов реализован дополнительный механизм оценки качества. Для каждого кандидата строился список ближайших слов, после чего анализировалось качество полученного ранжирования: наличие шумовых слов, естественность ассоциаций и пригодность слова для игрового сценария. Только после этого слово включалось в финальный список секретных.

На уровне серверной части итоговая система переработана под готовый словарь и заранее подготовленные векторы. При первом запуске приложения, если таблица Words пустая, происходила автоматическая загрузка слов из файла all-words.txt. Далее слова из secret-words.txt помечались как доступные для загадывания, а в базу данных сохранялись соответствующие векторы из модели sociation2vec800.vec.

После запуска новой игры система случайным образом выбирает слово из списка допустимых секретных слов. Для выбранного слова формируется карта рангов. Для этого вектор загаданного слова сравнивается со всеми словами общего словаря по косинусной близости.

После вычисления значений все слова сортируются по убыванию близости. Первое место занимает само загаданное слово, далее следуют наиболее близкие по смыслу варианты. Позиция слова в данном отсортированном списке и является его рангом.

Для повышения производительности сформированная карта рангов сохраняется в кэш памяти сервера. Это позволяет не пересчитывать сравнение всех слов заново при каждой попытке пользователя и значительно ускоряет обработку запросов.

Когда пользователь вводит слово, система проверяет его наличие в словаре, после чего получает заранее рассчитанный ранг и возвращает результат клиентской части. Если пользователь ввел само загаданное слово, игра считается завершенной победой.

В одиночном режиме игрок самостоятельно пытается найти правильный ответ, анализируя ранги предыдущих попыток (Рисунок 3.9). В многопользовательском режиме используется тот же принцип, однако попытки отправляются двумя участниками одновременно, а состояние игры синхронизируется через SignalR. Побеждает игрок, первым угадавший слово.

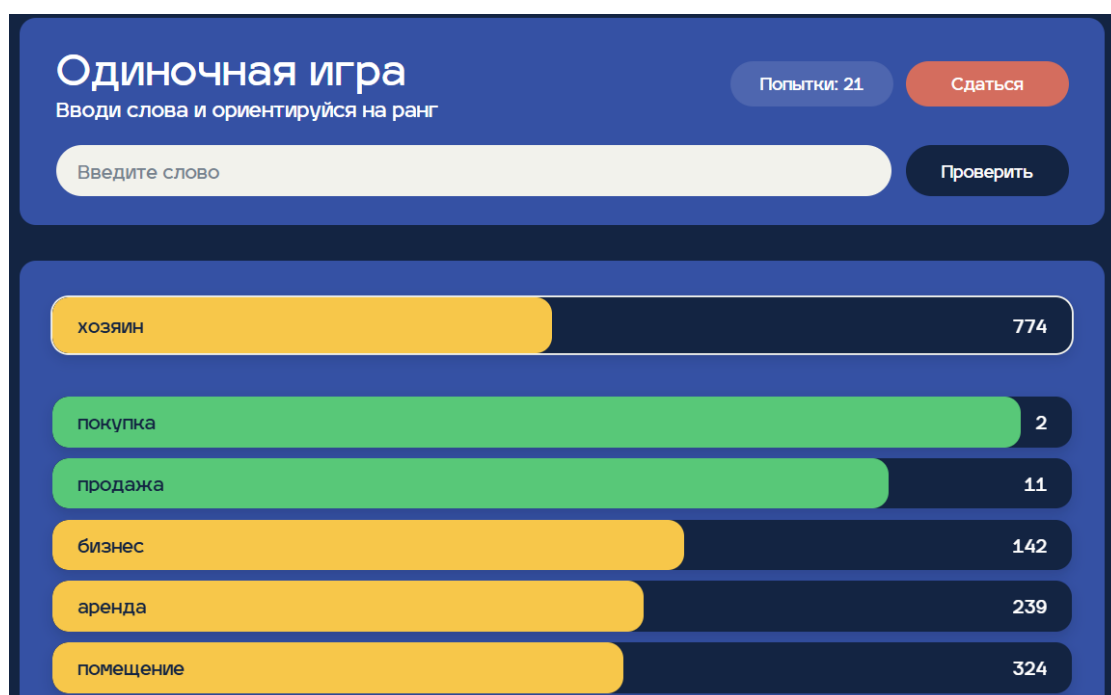


Рисунок 3.9 Одиночный режим игры

В процессе разработки система рангов несколько раз перерабатывалась. Итоговое решение показало, что качество игры определяется сочетанием трех факторов: правильным источником ассоциаций, качественной фильтрацией общего словаря и строгим отбором секретных слов.

В итоге игровая логика GuessWord реализована как система семантического поиска, адаптированная под игровой сценарий. Это позволило получить ранги, которые работают не только математически корректно, но и воспринимаются пользователем естественно в процессе угадывания слов.

3.6 Развертывание веб-приложения

В рамках разработки веб-приложения GuessWord выполнен процесс развертывания проекта в сети Интернет. Целью данного этапа являлось обеспечение постоянного доступа к приложению через браузер с любого устройства без необходимости локального запуска проекта на компьютере разработчика.

Для размещения приложения использован удаленный сервер VDS сервиса Selectel [13] с операционной системой Ubuntu 22.04 (Рисунок 3.10). Сервер получил постоянный публичный IP-адрес, что позволило открыть доступ к приложению из сети Интернет. Управление сервером выполнялось через удаленное подключение по протоколу SSH.

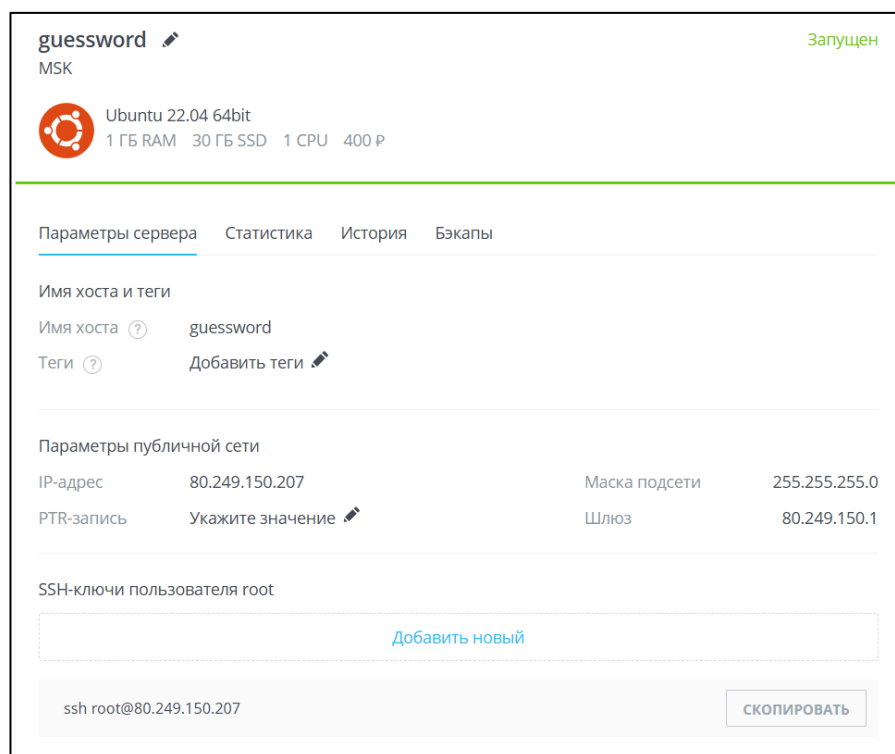


Рисунок 3.10 Информация о удаленном сервере

На сервере установлены: Docker, docker-compose и средства настройки сетевого доступа [10]. Использование Docker позволило запускать отдельные части проекта в изолированных контейнерах, что упростило перенос приложения на сервер и обеспечило стабильность окружения.

Для хранения конфиденциальных параметров окружения создан файл `.env`, содержащий настройки базы данных и секретный ключ JWT. Такой подход позволяет не размещать чувствительные данные в исходном коде проекта.

Для серверной части подготовлен `Dockerfile`, описывающий процесс сборки и публикации приложения `GuessWord.Api`. Сборка выполняется в образе с `.NET SDK`, после чего опубликованные файлы переносятся в runtime-образ `ASP.NET Core`, предназначенный для запуска приложения.

Для запуска всей системы использовался файл `docker-compose.server.yml`. В нем настроены три основных сервиса: контейнер базы данных `PostgreSQL`, контейнер серверного приложения `API` и контейнер `nginx` [11]. Контейнер базы данных использует постоянное хранилище, контейнер `API` подключается к базе данных и получает переменные окружения, а `nginx` выполняет функции веб-сервера и обратного прокси.

Отдельно настроена конфигурация `nginx`. Все запросы пользователя к основному адресу приложения направляются на клиентскую часть, а обращения по маршрутам `/api/` и `/hubs/` перенаправляются в серверную часть `ASP.NET Core`. Благодаря этому клиент и сервер работают через единый адрес сайта.

Для автоматизации публикации проекта применялся сервис `GitHub Actions` [12]. В репозитории настроен сценарий `deploy.yml`, который запускался при отправке изменений в ветку `main`. В процессе выполнения сценария происходила сборка клиентской части, копирование файлов проекта на сервер и последующий перезапуск контейнеров. Это позволило настроить автоматическое обновление проекта после загрузки изменений в репозиторий.

В результате выполненной работы веб-приложение `GuessWord` успешно опубликовано в сети Интернет и стало доступно пользователям через браузер по публичному адресу сервера. Развертывание позволило перевести проект из локальной стадии разработки в полноценно работающий сетевой сервис с возможностью дальнейшего сопровождения и обновления.

4 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

4.1 Использование программы

Разработанное веб-приложение GuessWord предназначено для игры в угадывание слов в одиночном и многопользовательском режимах. Для начала работы пользователю необходимо открыть сайт в браузере по адресу <http://80.249.150.207>. Приложение не требует установки дополнительного программного обеспечения и доступно с любого устройства, имеющего подключение к сети Интернет.

После загрузки сайта пользователю отображается главная страница приложения, с которой можно перейти к основным разделам системы: авторизации, регистрации, игровым режимам, истории игр и таблице лидеров (Рисунок 4.1).

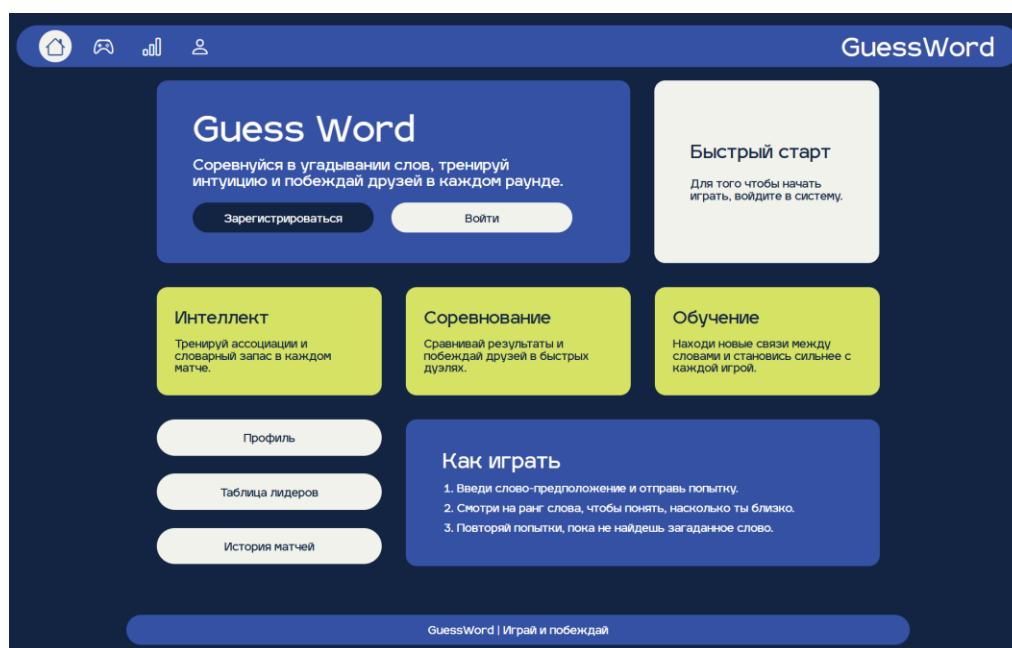


Рисунок 4.1 Главная страница

Для доступа к персональным функциям системы необходимо создать или войти в существующую учетную запись. Пользователь переходит на страницу авторизации или регистрации, вводит необходимые данные и отправляет форму (Рисунок 4.2).

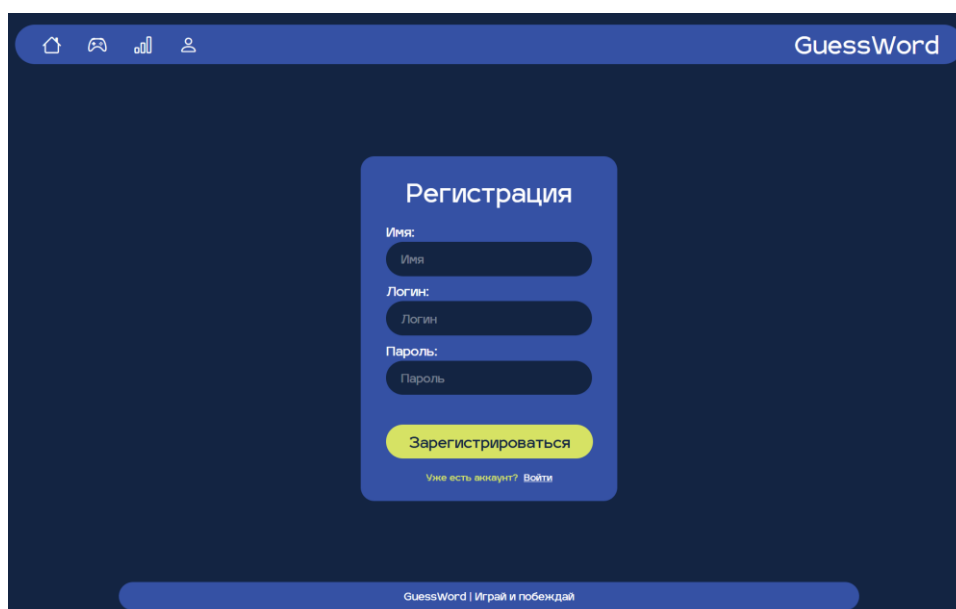


Рисунок 4.2 Страница регистрации

После успешного входа в систему пользователю открывается доступ к игровым возможностям приложения. (Рисунок 4.3).

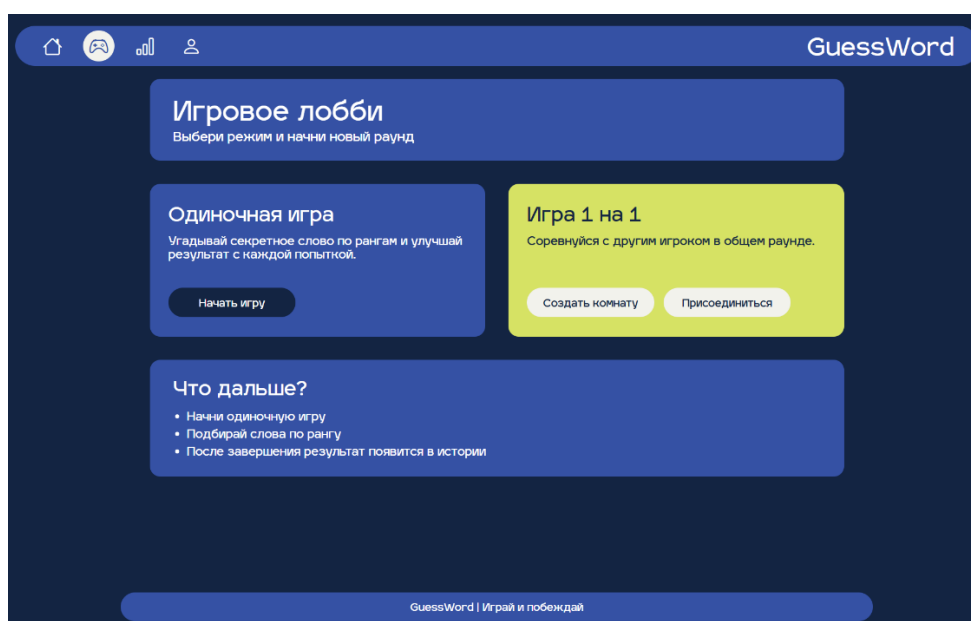


Рисунок 4.3 Страница выбора режима игры

В одиночном режиме система загадывает слово, после чего пользователь вводит варианты ответа в специальное поле. После каждой попытки отображается ранг введенного слова, показывающий степень близости к правильному ответу. Чем меньше значение ранга, тем ближе слово к загаданному. При вводе правильного слова партия завершается победой, а результат сохраняется в истории игр.

Для запуска многопользовательского режима пользователь может создать комнату либо подключиться к существующей по специальному коду. После подключения второго игрока создатель запускает матч. (Рисунок 4.4).

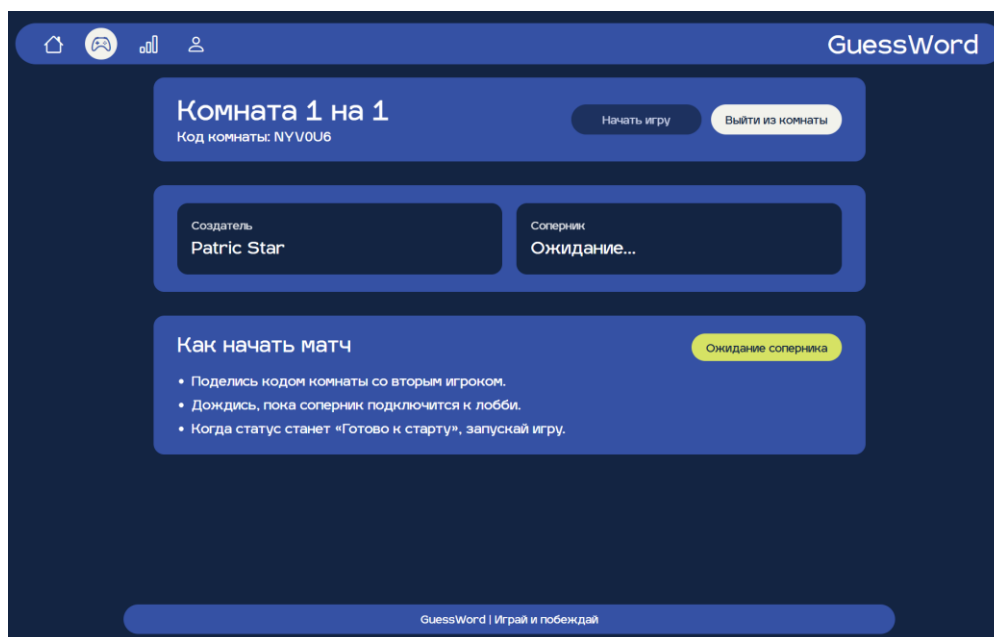


Рисунок 4.4 Комната многопользовательского режима

Во время многопользовательской игры действия участников синхронизируются в режиме реального времени. Победителем становится игрок, первым угадавший слово (Рисунок 4.4).



Рисунок 4.4 Игра 1 на 1

Дополнительно пользователю доступны страницы истории игр и таблицы лидеров.

История игр содержит сведения о завершенных матчах и результатах игрока. Таблица лидеров отображает рейтинг пользователей по накопленным достижениям.

В личном профиле пользователь может просматривать основные сведения об учетной записи, а также статистику сыгранных матчей, включая общее количество игр, число побед и процент успешных результатов (Рисунок 4.5). Наличие такого раздела обеспечивает быстрый доступ к персональным данным и сводной информации об игровой активности пользователя.

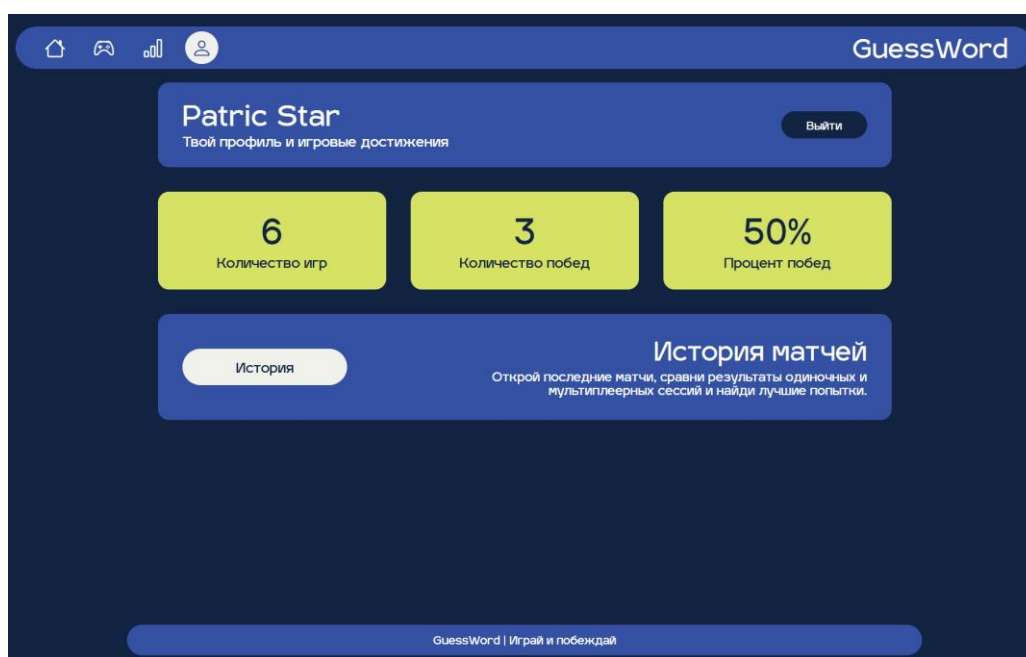


Рисунок 4.5 Профиль пользователя

Для завершения работы с приложением пользователь может выйти из учетной записи через интерфейс приложения. После этого доступ к персонализированным функциям системы ограничивается до повторного прохождения авторизации.

В итоге веб-приложение GuessWord предоставляет пользователю удобный доступ ко всем функциям системы через браузер: регистрацию, авторизацию, одиночную игру, многопользовательский режим, просмотр истории матчей и таблицы лидеров.

5 ТЕСТИРОВАНИЕ

5.1 Тестирование методом «черного ящика»

Тестирование методом «чёрного ящика» выполнялось со стороны пользователя через интерфейс веб-приложения GuessWord. Основной задачей являлась проверка корректности работы пользовательских функций, доступных после открытия сайта. Результаты тестирования методом «чёрного ящика» представлены в таблице 5.1.

Таблица 5.1 - Тест-кейсы методом «чёрного ящика»

№	Проверяемая функция	Действие	Ожидаемый результат	Полученный результат
1	Открытие сайта	Перейти по адресу приложения	Загружается главная страница	Успешно
2	Регистрация	Заполнить форму регистрации	Создается учетная запись	Успешно
3	Авторизация	Ввести логин и пароль	Выполняется вход в систему	Успешно
4	Профиль пользователя	Открыть страницу профиля	Отображаются данные игрока	Успешно
5	Одиночная игра	Запустить новую игру	Открывается игровой экран	Успешно
6	Проверка слова	Ввести слово-попытку	Отображается ранг слова	Успешно
7	Победа в игре	Ввести загаданное слово	Игра завершается победой, отображается результат	Успешно
8	Сдача в игре	Нажать кнопку завершения/сдачи	Игра завершается поражением, отображается результат	Успешно
9	Создание комнаты	Создать multiplayer-комнату	Генерируется код комнаты	Успешно
10	Подключение к комнате	Ввести код комнаты	Игрок подключается к комнате	Успешно
11	Multiplayer-игра	Начать матч вдвоем	Открывается общий игровой экран	Успешно
12	История игр	Открыть раздел истории	Отображаются завершенные матчи	Успешно
13	Таблица лидеров	Открыть рейтинг	Отображается список игроков	Успешно
14	Обновление страницы	Обновить сайт после входа	Пользователь остается в системе	Успешно

По результатам тестирования методом «чёрного ящика» можно сделать вывод, что основные пользовательские функции приложения работают корректно. Система правильно обрабатывает действия пользователя и выводит ожидаемый результат.

5.2 Тестирование методом «белого ящика»

Тестирование методом «белого ящика» выполнялось с учетом внутренней структуры программы. Проверялась работа серверной части приложения, обработка запросов, авторизация, игровая логика и взаимодействие с базой данных. Результаты тестирования методом «белого ящика» представлены в таблице 5.2.

Таблица 5.2 - Тест-кейсы методом «белого ящика»

№	Проверяемая функция	Действие	Ожидаемый результат	Полученный результат
1	UserController	Выполнить регистрацию пользователя	Пользователь сохраняется в БД	Успешно
2	AuthController	Выполнить вход пользователя	Возвращается JWT-токен	Успешно
3	JWT-сервис	Сформировать токен	Токен содержит данные пользователя	Успешно
4	LocalStorageService	Сохранить токен	Токен записывается в localStorage	Успешно
5	AuthService клиента	Восстановить вход после обновления	Авторизация сохраняется	Успешно
6	GameService	Запустить одиночную игру	Создается новая партия	Успешно
7	RankingService	Рассчитать ранг слова	Возвращается позиция в рейтинге	Успешно
8	Cache	Повторно получить ranking map	Используются кэшированные данные	Успешно
9	GameService	Сохранить попытку игрока	Попытка записывается в БД	Успешно
10	RoomService	Создать комнату	Комната создается с кодом	Успешно
11	RoomService	Подключить второго игрока	Игрок добавляется в комнату	Успешно
12	SignalR Hub	Передать обновление комнаты	Клиенты получают событие	Успешно
13	MultiplayerGame	Обработать ход игрока	Состояние игры обновляется	Успешно
14	HistoryService	Получить историю игр	Возвращаются матчи пользователя	Успешно
15	LeaderboardService	Получить рейтинг игроков	Возвращается таблица лидеров	Успешно

Результаты тестирования методом «белого ящика» показали, что внутренняя логика системы работает корректно. Контроллеры принимают запросы, сервисы выполняют необходимые операции, а база данных корректно сохраняет и возвращает информацию.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была достигнута поставленная цель - разработано веб-приложение GuessWord, предназначенное для угадывания слов в одиночном и многопользовательском режимах. В процессе работы были решены все поставленные задачи. Проведен анализ предметной области словесных онлайн-игр, что позволило определить особенности и требования к системе и сформировать функциональные требования.

Была спроектирована структура базы данных, обеспечивающая хранение информации о пользователях, игровых сессиях, комнатах и результатах матчей, а также разработана архитектура клиентской и серверной частей приложения. Реализован пользовательский интерфейс с использованием Blazor WebAssembly и механизм регистрации и авторизации пользователей на основе JWT-токенов.

В рамках работы реализованы основные функциональные возможности: одиночный режим игры, многопользовательский режим с использованием комнат и технологии SignalR, хранение истории игр и таблица лидеров. Особое внимание уделено механизму подбора слов на основе векторных представлений (embeddings) и косинусной близости, что повышает интеллектуальную составляющую игрового процесса.

Также проведено тестирование приложения с устранением выявленных ошибок, что обеспечило стабильную работу системы. Выполнено развертывание на удаленном сервере, благодаря чему GuessWord доступен через браузер без необходимости установки.

Таким образом, все поставленные задачи решены, а цель курсовой работы достигнута. Разработанное веб-приложение GuessWord представляет собой полноценный программный продукт и может быть расширено за счет добавления новых игровых режимов, статистики, достижений и мобильной адаптации.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Троелсен Э. Язык программирования C# и платформа .NET. – СПб.: Питер, 2021.
2. Microsoft. ASP.NET Core Documentation [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/aspnet/core>
3. Microsoft. Blazor Documentation [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/aspnet/core/blazor>
4. PostgreSQL Documentation [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs>
5. Microsoft. Entity Framework Core Documentation [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ef/core>
6. Microsoft. SignalR Documentation [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/aspnet/core/signalr>
7. JSON Web Token (JWT) [Электронный ресурс]. – Режим доступа: <https://jwt.io>
8. Microsoft. .NET Documentation [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/dotnet>
9. Figma [Электронный ресурс]. – Режим доступа: <https://www.figma.com>
10. Visual Studio [Электронный ресурс]. – Режим доступа: <https://visualstudio.microsoft.com>
11. Docker Documentation [Электронный ресурс]. – Режим доступа: <https://docs.docker.com>
12. GitHub Actions Documentation [Электронный ресурс]. – Режим доступа: <https://docs.github.com/actions>
13. Selectel. Облачная инфраструктура и VDS [Электронный ресурс]. – Режим доступа: <https://selectel.ru>
14. sociation.org. Ассоциативный словарь слов [Электронный ресурс]. – Режим доступа: <https://sociation.org>
15. Pgvector Documentation [Электронный ресурс]. – Режим доступа: <https://github.com/pgvector/pgvector>

ПРИЛОЖЕНИЕ А

Техническое задание

1. Общие сведения

1.1. Наименование системы

1.1.1. Полное наименование системы

Полное наименование: программный продукт «WEB API и веб-приложение GuessWord».

1.1.2. Краткое наименование системы

Краткое наименование: GuessWord.

1.2. Основания для проведения работ

Работа выполняется на основании задания на курсовое проектирование.

1.3. Наименование организаций – Заказчика и Разработчика

1.3.1. Заказчик

Заказчик: ГАПОУ МЦК-КТИТС

Адрес: г. Казань, ул. Бари Галеева, д.3а

1.3.2. Разработчик

Разработчик: Смирнов Владимир Викторович

Адрес: г. Казань.

1.4. Плановые сроки начала и окончания работ

Сроки выполнения: (укажи свои даты, например)

16.02.2026 – 05.05.2026

1.5. Порядок оформления и предъявления результатов работ

Работы выполняются поэтапно в соответствии с календарным планом.

По завершении каждого этапа предоставляется отчетная документация.

2. Назначение и цели создания системы

2.1. Назначение системы

Программный продукт предназначен для реализации веб-приложения интеллектуальной игры GuessWord и обработки запросов пользователей.

2.2. Цели создания системы

Система создается с целью:

- обеспечения обработки игровых данных;
- реализации одиночного и многопользовательского режимов;
- хранения и обработки пользовательских данных;
- повышения удобства доступа к игре через веб-интерфейс;
- обеспечения надежности и безопасности системы;
- реализации логики проверки и ранжирования слов.

3. Характеристика объектов автоматизации

Таблица 3.1 – Характеристика объектов автоматизации

Подразделение	Процесс	Возможность автоматизации
Пользователи	Игра (угадывание слов)	Автоматизировано
Пользователи	Регистрация и авторизация	Автоматизировано
Игроки	Многопользовательская игра (1 на 1)	Автоматизировано
Система	Обработка и ранжирование слов	Автоматизировано
Система	Хранение истории игр	Автоматизировано
Система	Формирование таблицы лидеров	Автоматизировано

4. Требования к системе

4.1. Требования к системе в целом

4.1.1. Требования к структуре и функционированию

Система должна быть реализована как клиент-серверное приложение и включать следующие компоненты:

- клиентская часть (Blazor WebAssembly);
- серверная часть (ASP.NET Core Web API);
- база данных (PostgreSQL).

Система должна обеспечивать:

- хранение данных в централизованной базе данных;
- взаимодействие клиента и сервера через HTTP API;
- обработку пользовательских запросов;
- обмен данными в реальном времени (SignalR).

Режимы функционирования:

- основной режим – выполнение всех функций системы;
- режим обслуживания – обновление системы и исправление ошибок.

4.1.2. Требования к численности и квалификации персонала системы и режиму его работы

4.1.2.1. Требования к численности персонала

Для обеспечения функционирования системы GuessWord необходимо участие следующих пользователей:

- администратор системы – 1 человек;
- пользователь (игрок) – не ограничено.

Функциональные обязанности пользователей системы:

- администратор системы обеспечивает общее управление системой, выполняет настройку и администрирование данных, осуществляет контроль корректности информации, управляет пользователями и игровыми данными;
- пользователь (игрок) осуществляет регистрацию в системе, выполняет вход, участвует в игровом процессе, вводит слова, просматривает результаты игр, статистику и рейтинг.

4.1.3. Показатели назначения

4.1.3.1. Требования к сохранению работоспособности системы в различных вероятных условиях

В зависимости от различных вероятных условий система должна выполнять требования, приведенные в таблице 4.4.

Таблица 4.4 - Требования, выполняемые системой

Вероятное условие	Требование
Выход из строя сервера	Уведомление администратора системы
Потеря соединения клиента	Возможность восстановления игровой сессии
Обновление страницы пользователем	Сохранение текущего состояния игры
Ошибка ввода данных	Корректное уведомление пользователя

5. Требования к составу и содержанию работ по подготовке объекта автоматизации к вводу системы в действие

5.1. Технические мероприятия

До ввода системы в эксплуатацию должны быть выполнены следующие работы:

- установка и настройка серверной части приложения;
- установка необходимого программного обеспечения (платформа .NET, СУБД PostgreSQL);

- настройка базы данных и выполнение миграций;
- настройка параметров подключения к базе данных;
- развертывание клиентской части приложения;
- настройка взаимодействия клиентской и серверной частей;
- настройка обмена данными в реальном времени (SignalR);
- проверка корректности работы системы.

5.2. Организационные мероприятия

Перед вводом системы в эксплуатацию должны быть выполнены следующие организационные мероприятия:

- определение ответственного лица за администрирование системы;

- обеспечение доступа пользователей к системе;
- настройка ролей пользователей (администратор, пользователь);
- подготовка тестовых данных (пользователи, игры, статистика);
- проведение тестирования работы системы

5.3. Изменения в информационном обеспечении

Для обеспечения функционирования системы необходимо:

- создание и заполнение базы данных начальными данными;
- организация хранения информации о пользователях, играх, попытках и результатах;
- формирование структуры словаря слов;
- обеспечение актуальности и корректности данных.

При необходимости структура данных может изменяться в процессе разработки и сопровождения системы.

6. Требования к документированию

1) согласованный Разработчиком и Заказчиком перечень подлежащих разработке комплектов и видов документов, соответствующих требованиям ГОСТ 34.201-89 и НТД отрасли Заказчика приведена в таблице 6.1.

Таблица 6.1 – Перечень документов

Этап	Документы
Проектирование	Ведомость эскизного проекта
	Пояснительная записка
Разработка	Руководство пользователя
	Технологическая инструкция
	Инструкция по ведению базы данных
Ввод в эксплуатацию	Программа и методика испытаний
	Акт приёмки

	Протокол испытаний
--	--------------------

Документация оформляется в печатном и электронном виде (Word).

7. Источники разработки

Настоящее Техническое Задание разработано на основе следующих документов и информационных материалов:

- ГОСТ 24.701-86 «Надежность автоматизированных систем управления»;
- ГОСТ 15150-69 «Машины, приборы и другие технические изделия.

Исполнения для различных климатических районов. Категории, условия эксплуатации, хранения и транспортирования в части воздействия климатических факторов внешней среды»;

- ГОСТ 21958-76 «Система "Человек-машина". Зал и кабины операторов.

Взаимное расположение рабочих мест. Общие эргономические требования»;

- ГОСТ 12.1.004-91 «ССБТ. Пожарная безопасность. Общие требования»;
- ГОСТ 34.602-89 «Техническое задание на создание автоматизированной

системы»

- ГОСТ 19.201-78 «Техническое задание. Требования к содержанию и оформлению».

ПРИЛОЖЕНИЕ В

В данном приложении представлены основные фрагменты программного кода, отражающие ключевую логику веб-приложения **GuessWord**. Полный исходный код проекта находится в архиве с программным продуктом. В приложение включены наиболее значимые участки: настройка серверной части, подключение базы данных, JWT-аутентификация, работа с сущностями, игровая логика, расчет рангов, работа комнат, SignalR и взаимодействие клиентской части с API.

B.1 Настройка серверной части приложения
Фрагмент файла Program.cs демонстрирует регистрацию контроллеров, SignalR, Swagger, кэша, контекста базы данных и сервисов, необходимых для работы приложения.

```
using GuessWord.Api.Data;
using GuessWord.Api.Hubs;
using GuessWord.Api.Interfaces;
using GuessWord.Api.Services;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.EntityFrameworkCore;
using Microsoft.IdentityModel.Tokens;
using Pgvector.EntityFrameworkCore;
using System.Text;
```

```
var builder =
WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddSignalR();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
builder.Services.AddMemoryCache(options =>
{
options.SizeLimit = 100;
});
```

```
builder.Services.AddDbContext<AppDbContext>
(options =>
options.UseNpgsql(

builder.Configuration.GetConnectionString("De
faultConnection"),
o => o.UseVector()),
ServiceLifetime.Scoped);
```

```
builder.Services.AddScoped<IUserService,
UserService>();
builder.Services.AddScoped<IJwtService,
JwtService>();
builder.Services.AddScoped<IGameService,
GameService>();
builder.Services.AddScoped<IRoomService,
RoomService>();
builder.Services.AddScoped<DictionaryImportS
ervice>();
builder.Services.AddScoped<IRankService,
RankService>();
```

B.2 Настройка JWT-аутентификации
В данном фрагменте показана настройка JWT-аутентификации. Она используется для проверки токена пользователя при выполнении защищенных запросов к API.

```
builder.Services.AddAuthentication(JwtBearerD
efaults.AuthenticationScheme)
.AddJwtBearer(options =>
{
options.TokenValidationParameters = new
TokenValidationParameters
{
ValidateIssuer = false,
ValidateAudience = false,
ValidateLifetime = true,
ValidateIssuerSigningKey = true,
IssuerSigningKey = new
SymmetricSecurityKey(
```

```
Encoding.UTF8.GetBytes(builder.Configuration
["Jwt:Key"]!))
});
builder.Services.AddAuthorization();
```

B.3 Маршрутизация, SignalR и импорт словаря
После настройки приложения подключаются CORS, аутентификация, авторизация, контроллеры и SignalR-хаб. Также при запуске выполняются миграции базы данных и импорт словаря.

```
var app = builder.Build();
```

```
if (app.Environment.IsDevelopment())
{
app.UseSwagger();
app.UseSwaggerUI();
app.UseHttpsRedirection();
}
```

```
app.UseCors("AllowBlazor");
```

```
app.UseAuthentication();
app.UseAuthorization();
```

```
app.MapControllers();
app.MapHub<GameHub>("/hubs/game");
```

```
using (var scope = app.Services.CreateScope())
{
var context =
scope.ServiceProvider.GetRequiredService<App
DbContext>();
await context.Database.MigrateAsync();
```

```
var importer =
scope.ServiceProvider.GetRequiredService<Dict
ionaryImportService>();
await importer.ImportAsync();
}
```

```
app.Run();
```

B.4 Контекст базы данных AppDbContext
Контекст базы данных содержит наборы DbSet для основных сущностей системы и настройки связей между таблицами через Fluent API.

```
public class AppDbContext : DbContext
{
public AppDbContext(DbContextOptions
options) : base(options)
{
}
```

```
public DbSet<User> Users { get; set; }
public DbSet<Room> Rooms { get; set; }
public DbSet<Game> Games { get; set; }
public DbSet<GamePlayer> GamePlayers {
get; set; }
public DbSet<GameAttempt> GameAttempts
{ get; set; }
public DbSet<Word> Words { get; set; }
```

```
protected override void
OnModelCreating(ModelBuilder modelBuilder)
{
base.OnModelCreating(modelBuilder);
```

```
modelBuilder.Entity<User>()
.HasIndex(x => x.Login)
.IsUnique();
```

```
modelBuilder.Entity<User>()
.HasIndex(x => x.Name)
.IsUnique();
```

```
modelBuilder.Entity<Room>()
.HasIndex(x => x.Code)
.IsUnique();
```

```
modelBuilder.Entity<Word>()
.HasIndex(x => x.Text)
.IsUnique();
```

```
modelBuilder.Entity<Word>()
.Property(x => x.Embedding)
.HasColumnType("vector(800)");
}
```

B.5 Модель слова Word

Модель Word описывает слово из игрового словаря. Для каждого слова хранится текст, признак возможности использования в качестве загаданного слова и векторное представление.

```
public class Word
{
public int Id { get; set; }
public string Text { get; set; } = null!;
public bool CanBeSecret { get; set; }
public Vector? Embedding { get; set; }
public List<Game> SecretWordGames { get;
set; } = new();
}
```

B.6 Контроллер пользователя UserController
Контроллер пользователя отвечает за регистрацию, авторизацию и получение профиля. Основная логика вынесена в сервисный слой.

```
[ApiController]
[Route("api/[controller]")]
public class UserController : ControllerBase
{
private readonly IUserService _userService;
```

```
public UserController(IUserService
userService)
{
_userService = userService;
}
```

```
[HttpPost("register")]
public async Task<ActionResult>
Register(RegisterRequestDto request)
{
var result = await
_userService.Register(request);
return result is null ? BadRequest("He
удалось зарегистрироваться.") : Ok(result);
}
```

```
[HttpPost("login")]
public async Task<ActionResult>
Login(LoginRequestDto request)
{
var result = await
_userService.Login(request);
return result is null ?
BadRequest("Неверный логин или пароль.") :
Ok(result);
}
```

```
[Authorize]
[HttpGet("profile")]
public async Task<ActionResult>
GetProfile()
{
var claim =
User.FindFirst(ClaimTypes.NameIdentifier)?.Va
lue;
```

```
if (string.IsNullOrEmpty(claim) ||
!int.TryParse(claim, out var userId))
return Unauthorized();
```

```
var result = await
```

```

_userService.GetProfileAsync(userId);
    return result is null ? NotFound() :
Ok(result);
}
}
B.7 Сервис регистрации и авторизации
UserService
Сервис UserService выполняет проверку
данных пользователя, регистрацию,
хеширование пароля, вход в систему и
генерацию JWT-токена.
public async Task<AuthResponseDto?>
Register(RegisterRequestDto request)
{
    if (string.IsNullOrEmpty(request.Login)
||
string.IsNullOrEmpty(request.Password) ||
string.IsNullOrEmpty(request.Name))
        return null;

    var normalizedLogin = request.Login.Trim();
    var loginExists = await
_context.Users.AnyAsync(x => x.Login ==
normalizedLogin);
    if (loginExists)
        return null;

    var normalizedName = request.Name.Trim();
    var nameExists = await
_context.Users.AnyAsync(x => x.Name ==
normalizedName);
    if (nameExists)
        return null;

    var user = new User
    {
        Login = normalizedLogin,
        Name = normalizedName,
        CreatedAt = DateTime.UtcNow
    };

    user.PasswordHash =
_passwordHasher.HashPassword(user,
request.Password);

    _context.Users.Add(user);
    await _context.SaveChangesAsync();

    var token = _jwtService.GenerateToken(user);

    return new AuthResponseDto
    {
        UserId = user.Id,
        Token = token,
        Login = user.Login,
        Name = user.Name
    };
}
public async Task<AuthResponseDto?>
Login(LoginRequestDto request)
{
    if (string.IsNullOrEmpty(request.Login)
|| string.IsNullOrEmpty(request.Password))
        return null;

    var normalizedLogin = request.Login.Trim();

    var user = await
_context.Users.FirstOrDefaultAsync(x =>
x.Login == normalizedLogin);
    if (user is null)
        return null;

    var result =
_passwordHasher.VerifyHashedPassword(user,
user.PasswordHash, request.Password);

    if (result ==
PasswordVerificationResult.Failed)
        return null;

    var token = _jwtService.GenerateToken(user);

    return new AuthResponseDto
    {
        UserId = user.Id,
        Token = token,
        Login = user.Login,
        Name = user.Name!
    };
}

```

```

};
}
B.8 Генерация JWT-токена
Фрагмент JwtService показывает создание
набора claims, формирование ключа подписи
и генерацию JWT-токена, который затем
используется клиентской частью для доступа
к защищенным методам API.
public string GenerateToken(User user)
{
    var key = _configuration["Jwt:Key"]!;

    var claims = new List<Claim>
    {
        new Claim(ClaimTypes.NameIdentifier,
user.Id.ToString()),
        new Claim(ClaimTypes.Name, user.Login)
    };

    var credentials = new SigningCredentials(
new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(
key)),
SecurityAlgorithms.HmacSha256
);

    var token = new JwtSecurityToken(
claims: claims,
expires: DateTime.UtcNow.AddHours(12),
signingCredentials: credentials
);

    return new
JwtSecurityTokenHandler().WriteToken(token);
}
B.9 Запуск одиночной игры
Фрагмент GameService показывает создание
новой одиночной игры. Система выбирает
случайное секретное слово, создает запись
игры и связывает ее с пользователем.
public async Task<SingleGameResponseDto>
StartSingleGameAsync(int userId)
{
    await using var transaction = await
_context.Database.BeginTransactionAsync();

    var currentPlayer = await
_context.GamePlayers
.Include(gp => gp.Game)
.FirstOrDefaultAsync(gp =>
gp.UserId == userId &&
gp.Game.Mode == GameMode.Single
&&
gp.Game.Status ==
GameStatus.InProgress);

    if (currentPlayer is not null)
        return await
BuildGameResponseAsync(currentPlayer.
GameId, userId);

    var secretWord = await
GetRandomSecretWordAsync();

    var game = new Game
    {
        Mode = GameMode.Single,
        Status = GameStatus.InProgress,
        SecretWordId = secretWord.Id
    };

    var gamePlayer = new GamePlayer
    {
        Game = game,
        UserId = userId,
        AttemptsCount = 0,
        Result = GamePlayerResult.Playing,
        IsActiveSingleGame = true
    };

    _context.Games.Add(game);
    _context.GamePlayers.Add(gamePlayer);

    await _context.SaveChangesAsync();
    await transaction.CommitAsync();
    await
_rankService.PrepareRankingAsync(secretWord
.Id);

    return await

```

```

BuildGameResponseAsync(game.Id, userId);
}
B.10 Обработка попытки игрока
Метод SubmitGuessAsync принимает слово
пользователя, проверяет его наличие в
словаре, рассчитывает ранг и сохраняет
попытку в базе данных.
public async Task<SingleGameResponseDto>
SubmitGuessAsync(int userId,
SubmitGuessRequestDto request)
{
    var game = await _context.Games
.Include(g => g.Players)
.Include(g => g.Attempts)
.Include(g => g.SecretWord)
.FirstOrDefaultAsync(g => g.Id ==
request.GameId);

    if (game is null)
        throw new Exception("Игра не найдена.");

    if (game.Mode != GameMode.Single)
        throw new Exception("Неверный режим
игры.");

    if (game.Status == GameStatus.Finished)
        throw new Exception("Игра уже
завершена.");

    var player = game.Players.FirstOrDefault(p
=> p.UserId == userId);
    if (player is null)
        throw new Exception("Игрок не найден в
этой игре.");

    var normalizedWord =
request.Word.Trim().ToLower();
    if
(string.IsNullOrEmpty(normalizedWord))
        throw new Exception("Слово пустое.");

    var word = await _context.Words
.AsNoTracking()
.FirstOrDefaultAsync(w => w.Text ==
normalizedWord);

    if (word is null)
    {
        var invalidAttempt = new
GameAttemptResponseDto
        {
            Word = normalizedWord,
            Rank = null,
            IsValid = false,
            IsRepeated = false,
            CreatedAt = DateTime.UtcNow
        };

        return await BuildGameResponseAsync(
game.Id,
userId,
transientLastAttempt: invalidAttempt);
    }

    var rank = await
_rankService.GetRankAsync(game.SecretWordId,
word.Id);

    var newAttempt = new GameAttempt
    {
        GameId = game.Id,
        UserId = userId,
        Word = normalizedWord,
        Rank = rank,
        IsValid = true
    };

    game.Attempts.Add(newAttempt);
    player.AttemptsCount++;

    if (word.Id == game.SecretWordId)
    {
        game.Status = GameStatus.Finished;
        game.WinnerUserId = userId;
        game.FinishedAt = DateTime.UtcNow;
        player.Result = GamePlayerResult.Won;
        player.IsActiveSingleGame = false;
    }

    await _context.SaveChangesAsync();
}

```

```

        return await
        BuildGameResponseAsync(game.Id, userId,
        new Attempt.Id, false);
    }
}
B.11 Расчет рангов и косинусной близости
Сервис RankService формирует карту рангов
для загаданного слова. Для этого вектор
секретного слова сравнивается со всеми
словами словаря по косинусной близости.
private async Task<Dictionary<int, int>>
BuildRankingAsync(int secretWordId)
{
    var secret = await _context.Words
        .AsNoTracking()
        .Where(w => w.Id == secretWordId)
        .Select(w => new
        {
            w.Id,
            w.Embedding
        })
        .FirstOrDefaultAsync();

    if (secret is null)
        throw new Exception("Секретное слово
        не найдено.");

    if (secret.Embedding is null)
        throw new Exception("У секретного слова
        отсутствует embedding.");

    var words = await _context.Words
        .AsNoTracking()
        .Where(w => w.Embedding != null)
        .Select(w => new
        {
            w.Id,
            w.Embedding
        })
        .ToListAsync();

    var secretVector =
        secret.Embedding.ToArray();

    var similarities = words
        .Select(w => new
        {
            w.Id,
            Score = CosineSimilarity(secretVector,
            w.Embedding!.ToArray())
        })
        .OrderByDescending(x => x.Score)
        .ToList();

    var rankMap = new Dictionary<int, int>();

    for (int i = 0; i < similarities.Count; i++)
    {
        rankMap[similarities[i].Id] = i;
    }

    return rankMap;
}
private float CosineSimilarity(float[] a, float[] b)
{
    float dot = 0;
    float normA = 0;
    float normB = 0;

    for (int i = 0; i < a.Length; i++)
    {
        dot += a[i] * b[i];
        normA += a[i] * a[i];
        normB += b[i] * b[i];
    }

    if (normA <= 0 || normB <= 0)
        return float.MinValue;

    return dot / (float)(Math.Sqrt(normA) *
    Math.Sqrt(normB));
}
B.12 Кэширование карты рангов
Для повышения производительности карта
рангов сохраняется в кэше памяти сервера.
Это позволяет не пересчитывать близость
всех слов при каждой попытке пользователя.
public async Task<int> GetRankAsync(int
secretWordId, int guessWordId)
{

```

```

        var rankMap = await
        GetOrBuildRankingAsync(secretWordId);

        return rankMap.TryGetValue(guessWordId,
        out var rank)
        ? rank
        : int.MaxValue;
    }
}
private async Task<Dictionary<int, int>>
GetOrBuildRankingAsync(int secretWordId)
{
    var cacheKey = $"ranking_{secretWordId}";

    if (_cache.TryGetValue(cacheKey, out
    Dictionary<int, int>? rankMap))
        return rankMap!;

    rankMap = await
    BuildRankingAsync(secretWordId);
    SetRankingCache(cacheKey, rankMap);

    return rankMap;
}
private void SetRankingCache(string cacheKey,
Dictionary<int, int> rankMap)
{
    _cache.Set(cacheKey, rankMap, new
    MemoryCacheEntryOptions
    {
        AbsoluteExpirationRelativeToNow =
        TimeSpan.FromHours(3),
        Size = 1
    });
}
B.13 Создание и подключение к комнате
Сервис RoomService отвечает за создание
комнаты, генерацию уникального кода и
подключение второго игрока.
public async Task<RoomResponseDto>
CreateRoomAsync(int userId)
{
    var room = new Room
    {
        Code = await
        GenerateUniqueCodeAsync(),
        HostUserId = userId,
        Status = RoomStatus.Waiting
    };

    _context.Rooms.Add(room);
    await _context.SaveChangesAsync();

    return await
    BuildRoomResponseAsync(room.Code, userId)
    ?? throw new
    InvalidOperationException("Failed to create
    room.");
}
public async Task<RoomResponseDto>
JoinRoomAsync(int userId, string code)
{
    var normalizedCode = NormalizeCode(code);
    if
    (string.IsNullOrEmpty(normalizedCode))
        return null;

    var room = await _context.Rooms
        .FirstOrDefaultAsync(r => r.Code ==
        normalizedCode);

    if (room is null)
        return null;

    if (room.HostUserId == userId)
        return await
        BuildRoomResponseAsync(room.Code, userId);

    if (room.Status != RoomStatus.Waiting &&
    room.Status != RoomStatus.Full)
        return null;

    if (room.GuestUserId.HasValue &&
    room.GuestUserId.Value != userId)
        return null;

    if (room.GuestUserId != userId)
    {
        room.GuestUserId = userId;

```

```

        room.Status = RoomStatus.Full;
        await _context.SaveChangesAsync();
    }
}
return await
BuildRoomResponseAsync(room.Code, userId);
}
B.14 SignalR-хаб GameHub
GameHub используется для подключения
игроков к группам комнат и игр. Это
необходимо для обновления состояния
многопользовательского режима в реальном
времени.
public class GameHub : Hub
{
    public async Task JoinRoomGroup(string
    roomCode)
    {
        var normalizedCode =
        NormalizeRoomCode(roomCode);
        await
        Groups.AddToGroupAsync(Context.Connection
        Id, $"room-{normalizedCode}");
    }

    public async Task LeaveRoomGroup(string
    roomCode)
    {
        var normalizedCode =
        NormalizeRoomCode(roomCode);
        await
        Groups.RemoveFromGroupAsync(Context.Con
        nectionId, $"room-{normalizedCode}");
    }

    public async Task JoinGameGroup(int
    gameId)
    {
        await
        Groups.AddToGroupAsync(Context.Connection
        Id, $"game-{gameId}");
    }

    public async Task LeaveGameGroup(int
    gameId)
    {
        await
        Groups.RemoveFromGroupAsync(Context.Con
        nectionId, $"game-{gameId}");
    }

    private static string
    NormalizeRoomCode(string roomCode)
    {
        return
        roomCode.Trim().ToUpperInvariant();
    }
}
B.15 Настройка клиентской части Blazor
WebAssembly
Фрагмент Program.cs клиентского проекта
показывает регистрацию HttpClient, настройк
API и сервисов, используемых в Blazor
WebAssembly.
using GuessWord.Client;
using GuessWord.Client.Services;
using Microsoft.AspNetCore.Components.Web;
using
Microsoft.AspNetCore.Components.WebAssem
bly.Hosting;

var builder =
WebAssemblyHostBuilder.CreateDefault(args);

builder.RootComponents.Add<App>("#app");
builder.RootComponents.Add<HeadOutlet>("he
ad::after");

var baseUrl = new
Uri(builder.HostEnvironment.BaseAddress);

builder.Services.AddScoped(sp => new
HttpClient
{
    BaseAddress = baseUrl
});

builder.Services.AddScoped(_ => new
ApiSettings
{

```

```

        _httpClient.DefaultRequestHeaders.Authorization =
            new AuthenticationHeaderValue("Bearer",
                authData.Token);

        _userService.SetUser(authData.UserId,
            authData.Token, authData.Login,
            authData.Name);
    }

    public async Task RestoreUser()
    {
        var userIdValue = await
            _localStorageService.GetItemAsync("userId");
        var token = await
            _localStorageService.GetItemAsync("token");
        var login = await
            _localStorageService.GetItemAsync("login");
        var name = await
            _localStorageService.GetItemAsync("name");

        if (int.TryParse(userIdValue, out var userId)
            &&
            !string.IsNullOrEmpty(token) &&
            !string.IsNullOrEmpty(login) &&
            !string.IsNullOrEmpty(name))
        {
            _httpClient.DefaultRequestHeaders.Authorization =
                new
                    AuthenticationHeaderValue("Bearer", token);

            _userService.SetUser(userId, token,
                login, name);
        }

        _userService.MarkInitialized();
    }
}

```

В.16 Авторизация на клиенте и localStorage
Клиентский AuthService сохраняет данные пользователя и JWT-токен в localStorage, а также добавляет токен в заголовок Authorization для последующих запросов.

```

private async Task
    SaveUserData(AuthResponseDto authData)
    {
        await
            _localStorageService.SetItemAsync("userId",
                authData.UserId.ToString());
        await
            _localStorageService.SetItemAsync("token",
                authData.Token);
        await
            _localStorageService.SetItemAsync("login",
                authData.Login);
        await
            _localStorageService.SetItemAsync("name",
                authData.Name);
    }

```

```

        _httpClient.DefaultRequestHeaders.Authorization =
            new AuthenticationHeaderValue("Bearer", token);

        _userService.SetUser(userId, token,
            login, name);
    }

    _userService.MarkInitialized();
}

```

В.17 Подключение клиентской части к SignalR
Фрагмент страницы Room.razor показывает подключение клиента к SignalR-хабу,

```

        _httpClient.DefaultRequestHeaders.Authorization =
            new AuthenticationHeaderValue("Bearer", token);

        _userService.SetUser(userId, token,
            login, name);
    }

    _userService.MarkInitialized();
}

```

В.16 Авторизация на клиенте и localStorage
Клиентский AuthService сохраняет данные пользователя и JWT-токен в localStorage, а также добавляет токен в заголовок Authorization для последующих запросов.

```

private async Task
    SaveUserData(AuthResponseDto authData)
    {
        await
            _localStorageService.SetItemAsync("userId",
                authData.UserId.ToString());
        await
            _localStorageService.SetItemAsync("token",
                authData.Token);
        await
            _localStorageService.SetItemAsync("login",
                authData.Login);
        await
            _localStorageService.SetItemAsync("name",
                authData.Name);
    }

```

В.17 Подключение клиентской части к SignalR
Фрагмент страницы Room.razor показывает подключение клиента к SignalR-хабу,

Представленные фрагменты кода демонстрируют основные принципы реализации GuessWord: разделение клиентской и серверной частей, использование JWT-аутентификации, работу с базой данных через Entity Framework Core, хранение векторных представлений слов, расчет рангов по косинусной близости, кэширование результатов и синхронизацию многопользовательского режима через SignalR.